

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology



Department of Computer Science & Engineering

24 Series

Course Code: CSE 1200

Course Title: Competitive Programming Sessional

Experiment No: 03

Experiment Title: Solutions to Lab Contest-02 and Additional Practice Problems

Submitted by:

Fardin Bin Aslam Numan

Roll: 2403179

Section: C

Session: 2024-25

Submitted to:

Md. Mazharul Islam

Lecturer

Dept. of Computer Science &
Engineering, RUET

Date of Experiment: 10/01/2026

Date of Submission: 24/01/2026

1. Maximum GCD (GCD)

Problem Statement:

Given a number n , consider all integers from 1 to n . Among all pairs of distinct integers (a, b) , find the maximum possible greatest common divisor (GCD). For multiple test cases, return the maximum GCD for each n

Theory/Approach:

The maximum GCD of two numbers in the range $1 \dots n$ is always $n/2$ (integer division). This is because the pair $(n/2, n)$ gives the highest common divisor: n is divisible by $n/2$. Therefore, for each test case, we can directly return $n/2$ without checking all pairs, which is a simple greedy observation based on number theory

Code:

```
g++ gcd_maximumGCD.cpp > ...
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      int t;
7      cin >> t;
8      while (t--)
9      {
10         int n;
11         cin >> n;
12         cout << n / 2 << "\n";
13     }
14     return 0;
15 }
```

Output:

```
2 3 5
1
2
```

Verdict:

359200884	Jan/22/2026 18:30 UTC+6	fardinnuman	1370A - Maximum GCD	C++17 (GCC 7-32)	Accepted	31 ms	0 KB
-----------	-------------------------	-------------	---------------------	------------------	----------	-------	------

Discussion:

- Time Complexity: $O(t)$, since we calculate $n/2$ for each of the t test cases
- Space Complexity: $O(1)$, as no extra space is needed beyond input variables

2. Minimum LCM (LCM)

Problem Statement:

Given an integer n , find two positive integers a and b such that $a + b = n$ and the least common multiple (LCM) of a and b is minimized. If multiple pairs give the same minimum LCM, any pair can be output

Theory/Approach:

- The LCM of two numbers a and b is minimized when a and b share the maximum possible common factor
- To achieve this, we look for the largest divisor of n (less than n) as a , and set $b = n - a$.
- If n is even, the best pair is $(n/2, n/2)$, giving $\text{LCM} = n/2$
- If n is odd, we iterate through divisors up to $\sqrt{n}$ to find the divisor d that minimizes $\text{LCM}(d, n-d)$ efficiently

This reduces the brute-force check of all pairs to just checking divisors

Code:

```
G++ lcm_minimumLCM.cpp > ...
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      int t;
7      cin >> t;
8      while (t--)
9      {
10         long long n;
11         cin >> n;
12         long long a = 1;
13         for (long long i = 1; i * i <= n; i++)
14         {
15             if (n % i == 0)
16             {
17                 long long d1 = i, d2 = n / i;
18                 if (d1 <= n / 2)
19                     a = max(a, d1);
20                 if (d2 <= n / 2)
21                     a = max(a, d2);
22             }
23         }
24         cout << a << " " << n - a << "\n";
25     }
26 }
```

Output:

```
4 2 9 5 10
1 1
3 6
1 4
5 5
```

Verdict:

359201343	Jan/22/2026 18:34 UTC+6	fardinnuman	1765M - Minimum LCM	C++17 (GCC 7-32)	Accepted	62 ms	0 KB
-----------	-------------------------	-------------	---------------------	------------------	----------	-------	------

Discussion:

- Time Complexity: $O(\sqrt{n})$ per test case
- Space Complexity: $O(1)$, as only a few variables are needed for computation

3. Panoramix's Prediction (Prime)

Problem Statement:

Given two integers n and m (with $n < m$), where n is guaranteed to be a prime number:

- Determine if m is the next prime number after n
- Print YES if m is the next prime, otherwise NO

Theory/Approach:

- Iterate from $n + 1$ upwards and find the first prime number greater than n
- Compare it with m ...
 - If equal ... output YES
 - Otherwise ... output NO
- A prime-checking function can test divisibility from 2 up to $\sqrt{x}$
- Since $n, m \leq 50$, brute-force prime checking is efficient

Code:

```
G++ prime_Panoramix.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  bool isPrime(int x)
5  {
6      if (x < 2)
7          return false;
8      for (int i = 2; i < x; i++)
9      {
10         if (x % i == 0)
11             return false;
12     }
13     return true;
14 }
15
16 int main()
17 {
18     int n, m;
19     cin >> n >> m;
20
21     int nextPrime = n + 1;
22     while (!isPrime(nextPrime))
23     {
24         nextPrime++;
25     }
26
27     if (nextPrime == m)
28     {
29         cout << "YES" << endl;
30     }
31     else
32     {
33         cout << "NO" << endl;
34     }
35
36     return 0;
37 }
```

Output:

```
3 5  
YES
```

```
7 11  
YES
```

```
7 9  
NO
```

Verdict:

359202522	Jan/22/2026 18:43 ^{UTC+6}	fardinnuman	80A - Panoramix's Prediction	C++17 (GCC 7-32)	Accepted	92 ms	0 KB
-----------	------------------------------------	-------------	------------------------------	------------------	----------	-------	------

Discussion:

- Time Complexity: $O(\sqrt{m})$ for prime checking
- Space Complexity: $O(1)$, only a few variables are needed

4. Petya and Strings (Vector)

Problem Statement:

Petya has two strings of equal length consisting of uppercase and lowercase Latin letters. He wants to compare them lexicographically (dictionary order) while ignoring case. Your task is to determine if the first string is smaller, larger, or equal to the second string when compared case-insensitively

Theory/Approach:

To solve this, we convert both strings to the same case (either all lowercase or all uppercase) and then compare them character by character from left to right. If we find the first differing character, we determine the order based on which character comes first in the alphabet. If all characters are the same after conversion, the strings are equal. This approach works because lexicographical order compares strings based on the first position where they differ, and converting all letters to the same case ensures case-insensitive comparison

Code:

```
G++ vector_PetyaAndStrings.cpp > ...
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      string s1, s2;
7      cin >> s1 >> s2;
8
9      vector<char> v1, v2;
10
11     for (char c : s1)
12         v1.push_back(tolower(c));
13     for (char c : s2)
14         v2.push_back(tolower(c));
15
16     if (v1 < v2)
17         cout << -1 << "\n";
18     else if (v1 > v2)
19         cout << 1 << "\n";
20     else
21         cout << 0 << "\n";
22
23     return 0;
24 }
```

Output:

aaaa	abs	abcdefg
aaaA	Abz	AbCdEfF
0	-1	1

Verdict:

359204689	Jan/22/2026 18:59 ^{UTC+6}	fardinnuman	112A - Petya and Strings	C++17 (GCC 7-32)	Accepted	92 ms	100 KB
---------------------------	------------------------------------	-------------	--	------------------	----------	-------	--------

Discussion:

- Time Complexity: $O(n)$, where n is the length of the strings, because we traverse both strings once
- Space Complexity: $O(1)$ additional space if we convert in place, otherwise $O(n)$ if creating new strings for case conversion

5. Indian Summer (Map)

Problem Statement:

Alyona is collecting fallen leaves in a forest. Each leaf has a species and a colour. She only keeps a leaf if she hasn't already picked a leaf with the same species and colour combination. Given a list of leaves, determine how many unique leaves Alyona picks

Theory/Approach:

To solve this, we can track all the unique (species, colour) pairs Alyona picks. Every time we read a new leaf, we check whether this exact pair has been seen before. If not, we count it as a new leaf. A set (or hash table) is perfect for storing these pairs and checking uniqueness efficiently. This guarantees that each leaf counted is distinct in both species and colour, as required

Code:

```
G+ map_IndianSummer.cpp > ...
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int n;
6      cin >> n;
7
8      map<pair<string, string>, int> leafCount;
9
10     for (int i = 0; i < n; i++) {
11         string species, color;
12         cin >> species >> color;
13
14         leafCount[{species, color}]++;
15     }
16
17     cout << leafCount.size() << "\n";
18
19     return 0;
20 }
```

Output:

```
5
birch yellow
maple red
birch yellow
maple yellow
maple green
4
```

```
3
oak yellow
oak yellow
oak yellow
1
```

Verdict:

359206589	Jan/22/2026 19:12 UTC+6	fardinnuman	44A - Indian Summer	C++17 (GCC 7-32)	Accepted	62 ms	0 KB
---------------------------	-------------------------	-------------	-------------------------------------	------------------	----------	-------	------

Discussion:

- Time Complexity: $O(n)$, because each leaf is checked and inserted into the set once, and $n \leq 100$
- Space Complexity: $O(n)$, to store the unique species-colour pairs

6. Tom Riddle's Diary (Set)

Problem Statement:

Harry Potter wants to track all people who have possessed Tom Riddle's diary. Given a list of n names in order of possession, for each name, determine whether that person has possessed the diary before. If they have, print "YES"; otherwise, print "NO"

Theory/Approach:

We can keep a record of all names we have seen so far using a set (or hash table). As we read each name from the list, we check if it already exists in the set. If it does, we output "YES"; otherwise, "NO" and add the name to the set. This ensures that we efficiently track previous possession without repeatedly scanning the entire list

Code:

```
set_TomRiddle.cpp > ...
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main()
6  {
7      int n;
8      cin >> n;
9
10     set<string> seen;
11     for (int i = 0; i < n; i++)
12     {
13         string name;
14         cin >> name;
15         if (seen.count(name))
16         {
17             cout << "YES\n";
18         }
19         else
20         {
21             cout << "NO\n";
22             seen.insert(name);
23         }
24     }
25
26     return 0;
27 }
```

Output:

6	
tom	3
NO	a
lucius	
NO	NO
ginny	
NO	a
harry	
NO	YES
ginny	
YES	a
harry	
YES	YES

Verdict:

359203739	Jan/22/2026 18:52 UTC+6	fardinnuman	855A - Tom Riddle's Diary	C++17 (GCC 7-32)	Accepted	46 ms	100 KB
-----------	-------------------------	-------------	---------------------------	------------------	----------	-------	--------

Discussion:

- Time Complexity: $O(n)$, because each name is checked and inserted into the set once
- Space Complexity: $O(n)$, to store the unique names encountered

7. Is Your Horseshoe on the Other Hoof? (Pair)

Problem Statement:

Valera has four horseshoes, each with a certain colour. To be stylish, he wants all four horseshoes to have different colours. Given the colours of the four horseshoes he already has, determine the minimum number of horseshoes he needs to buy so that all four are distinct

Theory/Approach:

The solution is based on counting the number of unique colours among the four horseshoes. If all four colours are unique, Valera doesn't need to buy any. Otherwise, the number of horseshoes to buy is 4. A set is ideal for tracking unique colours because it automatically discards duplicates. By computing the size of this set, we immediately know how many horseshoes are missing to reach four distinct colours

Code:

```
G+ pair_horseshoe.cpp > ...
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      vector<int> colors(4);
6      for(int i = 0; i < 4; i++) {
7          cin >> colors[i];
8      }
9
10     vector<pair<int,int>> shoes;
11     for(int i = 0; i < 4; i++) {
12         shoes.push_back({colors[i], i});
13     }
14
15     sort(shoes.begin(), shoes.end());
16
17     int duplicates = 0;
18     for(int i = 0; i < 3; i++) {
19         if(shoes[i].first == shoes[i+1].first) {
20             duplicates++;
21         }
22     }
23
24     cout << duplicates << "\n";
25
26     return 0;
27 }
```

Output:

```
1 7 3 3
1
```

```
7 7 7 7
3
```

Verdict:

359209314	Jan/22/2026 19:31 UTC+6	fardinnuman	228A - Is your horseshoe on the other hoof?	C++17 (GCC 7-32)	Accepted	92 ms	0 KB
-----------	-------------------------	-------------	---	------------------	----------	-------	------

Discussion:

- Time Complexity: $O(1)$, since we only process four numbers.
- Space Complexity: $O(1)$, as we store at most four colours in the set.

8. Plug-in (Stack)

Problem Statement:

Polycarp often types letters twice by mistake. Given a string, repeatedly remove all consecutive pairs of identical letters until no such pairs remain. Output the final processed string

Theory/Approach:

We can efficiently solve this problem using a stack-like approach. Iterate through the string and for each character:

- If the stack is empty, push the character
- If the top of the stack is the same as the current character, pop the top (removing the pair)
- Otherwise, push the character onto the stack

This ensures all consecutive identical letters are removed, including any new pairs that form after removal. At the end, the stack contains the final processed string

Code:

```

G++ stack_Plugin.cpp > ...
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      string s;
7      cin >> s;
8
9      string result = "";
10
11     for (char c : s)
12     {
13         if (!result.empty() && result.back() == c)
14         {
15             result.pop_back();
16         }
17         else
18         {
19             result.push_back(c);
20         }
21     }
22
23     cout << result << "\n";
24     return 0;
25 }

```

Output:

```

hhoowaaaaareyyoouu
wre

```

```

reallazy
rezy

```

```

abacabaabacabaa
a

```

Verdict:

359205274	Jan/22/2026 19:03UTC+6	fardinnuman	81A - Plug-in	C++17 (GCC 7-32)	Accepted	124 ms	100 KB
---------------------------	------------------------	-------------	-------------------------------	------------------	----------	--------	--------

Discussion:

- Time Complexity: $O(n)$, because each character is pushed/popped at most once
- Space Complexity: $O(n)$, for the stack storing characters

9. Queue at the School (Queue)

Problem Statement:

Boys and girls are in a queue. Each second, any boy (B) who has a girl (G) immediately behind him swaps positions with her. Given the initial queue and a number of seconds t , determine the final arrangement of the queue after t seconds

Theory/Approach:

We can simulate the process directly. For each second from 1 to t , iterate through the queue from left to right. Whenever a B is immediately followed by a G, swap them and skip the next position to avoid double-swapping in the same second. After t iterations, the queue will reflect the final arrangement. This works efficiently because the maximum constraints are small ($n, t \leq 50$)

Code:

```
G+ queue_QueueAtTheSchool.cpp > ...
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int n, t;
6      cin >> n >> t;
7      string s;
8      cin >> s;
9
10     while(t--> {
11         for(int i = 0; i < n-1; i++) {
12             if(s[i] == 'B' && s[i+1] == 'G') {
13                 swap(s[i], s[i+1]);
14                 i++;
15             }
16         }
17     }
18
19     cout << s << "\n";
20     return 0;
21 }
```

Output:

5 1	5 2	4 1
BGGBG	BGGBG	GGGB
GBGGB	GGBGB	GGGB

Verdict:

359209647	Jan/22/2026 19:33UTC+6	fardinnuman	266B - Queue at the School	C++17 (GCC 7-32)	Accepted	92 ms	0 KB
---------------------------	------------------------	-------------	--	------------------	----------	-------	------

Discussion:

- Time Complexity: $O(n*t)$, as we iterate through the queue for each second
- Space Complexity: $O(n)$, for storing the queue

Contest Problems

A. Breaking the Records

Problem Statement:

Maria plays college basketball and records her points in each game. The first game sets her initial record for both maximum and minimum scores. For each subsequent game, she counts how many times she breaks her record for most points and least points during the season

Theory/Approach:

1. Initialize the maximum and minimum records with the first game's score
2. Iterate through the remaining scores:
 - If a score is greater than the current maximum, update maximum and increment maxCount
 - If a score is less than the current minimum, update minimum and increment minCount
3. Print the counts of maximum and minimum record breaks

Code:

```
G++ Acpp > ...
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int n;
7      cin >> n;
8
9      int score, max, min;
10     int maxCount = 0, minCount = 0;
11
12     cin >> score;
13     max = min = score;
14
15     for (int i = 1; i < n; i++)
16     {
17         cin >> score;
18         if (score > max)
19         {
20             max = score;
21             maxCount++;
22         }
23         else if (score < min)
24         {
25             min = score;
26             minCount++;
27         }
28     }
29
30     cout << maxCount << " " << minCount << endl;
31
32     return 0;
33 }
```

Output:

```
9
10 5 20 20 4 5 2 25 1
2 4
```

Verdict:

66876881	fardinnuman	A	Accepted	133	521	C++	12 days ago
----------	-------------	---	----------	-----	-----	-----	-------------

Discussion:

- **Time Complexity:** $O(1)$, only a few conditional checks
- **Space Complexity:** $O(1)$, uses a fixed number of integer variables

B. Library Fine

Problem Statement:

The problem is to calculate the library fine based on the actual return date and the expected return date of a book. If the book is returned on or before the due date, no fine is charged. If it is returned late within the same month and year, the fine is calculated based on the number of days late. If the return occurs in a later month but within the same year, the fine depends on the number of months late. If the book is returned in a later year than the due year, a fixed maximum fine is applied

Theory/Approach:

The solution works by comparing the return date with the due date in decreasing order of significance: year, month, and day. First, the years are compared to determine if the return is late by a year. If the years are the same, the months are compared to calculate a monthly fine. If both year and month match, the days are compared to calculate a daily fine. If none of these conditions apply, the fine is zero. This approach ensures that only the least precise measure of lateness is used, as required by the problem

Code:

```
G++ B.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  int fine(int day, int month, int year, int dayy, int monthh, int yearr)
5  {
6
7      if (year > yearr)
8      {
9          return 10000;
10     }
11     if (year == yearr && month > monthh)
12     {
13         return 500 * (month - monthh);
14     }
15     if (year == yearr && month == monthh && day > dayy)
16     {
17         return 15 * (day - dayy);
18     }
19     return 0;
20 }
21
22 int main()
23 {
24     int day, month, year;
25     int dayy, monthh, yearr;
26
27     cin >> day >> month >> year;
28     cin >> dayy >> monthh >> yearr;
29     cout << fine(day, month, year, dayy, monthh, yearr) << endl;
30
31     return 0;
32 }
```

Output:

```
9 6 2015
6 6 2015
45
```

Verdict:

66877134	fardinnuman	B	Accepted	147	576	C++	12 days ago
----------	-------------	---	----------	-----	-----	-----	-------------

Discussion:

- **Time Complexity:** $O(n)$, iterates through all scores once
- **Space Complexity:** $O(1)$, uses only a few integer variables

C. Strange Counter

Problem Statement:

The problem describes a strange counter that starts at a value of 3 and decreases by 1 every second until it reaches 1. After reaching 1, the counter resets to double the initial value of the previous cycle and repeats the process. Given a specific time t , the task is to determine the value displayed by the counter at that exact second

Theory/Approach:

The solution identifies which countdown cycle the given time t belongs to. Each cycle starts at a specific time and has a length that doubles every cycle. By iteratively subtracting completed cycle lengths from t , the correct cycle is found. Once the cycle is determined, the displayed value is calculated by subtracting the elapsed time within the cycle from the cycle's starting value. This avoids simulating every second and efficiently computes the result

Code:

```
G++ Cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  long long strangeCounter(long long t)
5  {
6      long long start = 1;
7      long long length = 3;
8      while (t > (start + length - 1))
9      {
10         start = start + length;
11         length = length * 2;
12     }
13     return length - (t - start);
14 }
15
16 int main()
17 {
18     long long t;
19     if (!(cin >> t))
20     {
21         return 0;
22     }
23     cout << strangeCounter(t) << endl;
24     return 0;
25 }
```

Output:

4
6

Verdict:

66876925	fardinnuman	C	Accepted	111	420	C++	12 days ago
----------	-------------	---	----------	-----	-----	-----	-------------

Discussion:

- **Time Complexity:** $O(\log t)$, because the cycle length doubles each iteration
- **Space Complexity:** $O(1)$, only a few variables are used

D. Ema's Supercomputer

Problem Statement:

This problem required identifying all possible valid plus shapes in a grid, calculating their areas, and then determining the maximum product of the areas of two non-overlapping pluses

Discussion:

Although the problem statement was clear, implementing an efficient and correct solution within the contest time proved difficult due to multiple conceptual and implementation challenges. The primary difficulty was generating all valid plus shapes of different sizes and accurately tracking their occupied cells to prevent overlap. Managing combinations of two pluses while ensuring they do not intersect significantly increased complexity. Additionally, handling grid boundaries and maintaining efficiency under time constraints made the problem challenging to complete during the lab contest